

■ How to use this guide

Each answer is crafted specifically from your TCS BaNCS experience, Health Transformer project, and Lovable Clone. Read each answer aloud 2–3 times before the interview. Personalise numbers and specifics where prompted.

1 · Introduction & Background

Q1. Walk me through your experience at TCS and how it prepares you for a Technology Architect role.

Answer: I joined TCS in November 2022 as a System Engineer and have spent 3.3+ years in the BFSI and Health Insurance domains. My work centred on TCS BaNCS — the core insurance and banking platform — and a health insurance product called Health Transformer. I worked across the Policy, Claims, and Workflow modules, which gave me end-to-end exposure to insurance business flows. On the technical side, I migrated legacy PL/SQL procedures to Java using JDBC, designed RESTful APIs, resolved EDI defects, and handled post-conversion production issues. These experiences have sharpened my ability to think about architecture — how components integrate, where bottlenecks occur, and how to design systems that are maintainable and scalable. My Lovable Clone personal project extended that thinking into modern distributed architecture with Kafka, Redis, Docker, and PostgreSQL.

■ *Interviewer tip: Be specific about modules and business context — generic answers will not impress.*

Q2. You have worked on BaNCS — describe the overall architecture of the platform and your role within it.

Answer: BaNCS is a multi-tier enterprise platform built on a Java/Oracle stack. It follows a layered architecture: a presentation layer (web/channel), a service/API layer for business logic, and a data layer backed by Oracle. The platform supports configurable product setups, policy lifecycle management, claims processing, and workflow orchestration. My role involved backend development and integration work — primarily in Java (Spring Boot), PL/SQL, and JDBC. I contributed to the Policy module by building APIs for historical policy data retrieval, and worked on channel-specific business logic for Health Transformer's Channel ID requirements. I also managed defect fixes in the EDI module and supported post-conversion stability for client server deliveries.

Q3. How did your work evolve from a developer to someone thinking about architectural decisions?

Answer: My shift toward architectural thinking happened gradually. Early on I was focused on executing tasks — migrating stored procedures, fixing defects. But as I worked on post-conversion production issues, I started asking 'why did this break?' rather than just 'how do I fix it?' That led me to understand root causes at a system level — data access patterns, transaction boundaries, service coupling. With Health Transformer, collaborating with Business Analysts on Channel ID requirements made me appreciate how business rules translate into architectural decisions. My Lovable Clone project was where I deliberately applied architectural patterns — CQRS thinking with JDBC/Spring, event-driven design with Kafka, and caching strategy with Redis — to solidify that mindset.

■ *Interviewer tip: Show the journey — growth from executor to architect is exactly what they want to hear.*

Q4. Tell me about the most complex technical problem you solved independently at TCS.

Answer: One of the most complex situations I handled was a post-conversion production issue where policy data was returning inconsistent results after a server migration. I performed root cause analysis and traced it to a legacy PL/SQL procedure that was making implicit assumptions about session-level Oracle settings that changed in the new environment. I re-engineered the procedure into a Java method using PreparedStatement, making all parameters explicit and removing environment dependencies. This not only fixed the immediate issue but improved maintainability. The challenge was the time pressure — it was a

client-facing system and I had to diagnose and resolve within critical timelines.

2 · Java Enterprise Edition (Must-Have)

Q5. What are the core pillars of Java EE? How have you used them in enterprise projects?

Answer: Java EE (now Jakarta EE) is built around several key specifications: Servlets and JSP for web tier, EJB for business components, JPA for persistence, JAX-RS for RESTful APIs, JMS for messaging, CDI for dependency injection, JTA for distributed transactions, and Java Security for authentication. In my TCS work I primarily used Spring Boot — which wraps many Java EE concepts — along with JDBC for data access (PreparedStatement, CallableStatement), and built RESTful APIs with Spring's JAX-RS-aligned approach. The core Java EE principles of component isolation, declarative transaction management, and container-managed services are ones I apply even within Spring's ecosystem.

Q6. Explain the difference between EJB, CDI, and Spring beans. Which did you use and why?

Answer: EJBs are Java EE-managed components with built-in transaction management, security, and remote calling capability — they live in an EJB container. CDI (Contexts and Dependency Injection) is a lighter standard for dependency injection within Java EE without the full EJB overhead. Spring beans are managed by the Spring IoC container and offer similar DI capabilities with extensive ecosystem support. At TCS I worked primarily with Spring Boot, so Spring beans via `@Component`, `@Service`, and `@Repository` were my daily tools. Spring was the pragmatic choice because BaNCS integrations used Spring's REST templates and transaction managers, and the team had established patterns around it. For a new greenfield Java EE system I would evaluate EJB if container-managed transactions and remote EJB calls were needed, otherwise CDI or Spring.

Q7. How did you use JDBC vs JPA/Hibernate in your PL/SQL-to-Java migration? What tradeoffs did you make?

Answer: In my migration at TCS, I used JDBC deliberately rather than JPA because the PL/SQL procedures being migrated had complex stored procedure calls, bulk operations, and Oracle-specific SQL that mapped poorly to JPA's ORM paradigm. JDBC with PreparedStatement and CallableStatement gave me precise control over the SQL being sent, which was critical for maintaining performance parity with the original procedures. JPA would have introduced an abstraction layer that could hide performance issues and made debugging harder. The tradeoff is more boilerplate code and manual ResultSet mapping, but for high-volume insurance data operations, explicit SQL control was the right call. In my Lovable Clone project I used JPA with PostgreSQL for standard CRUD operations where productivity mattered more than SQL control.

Q8. How do you handle transaction management in Java EE? Explain JTA vs local transactions with a real example.

Answer: Local transactions are managed by a single resource — for example, a JDBC connection's commit/rollback. JTA (Java Transaction API) manages transactions across multiple resources — such as a database and a JMS queue in a single atomic operation. In BaNCS, policy lifecycle operations often needed to update both the policy database and trigger workflow state changes atomically. If the workflow update failed, the policy change had to roll back. This is a JTA-style distributed transaction scenario. In Spring Boot, I used `@Transactional` which delegates to the configured transaction manager — typically the `DataSourceTransactionManager` for single-datasource setups. For multi-resource scenarios in enterprise settings, a JTA provider like Atomikos or the app server's built-in JTA implementation would be used.

Q9. What is the role of JAX-RS in building RESTful APIs? Walk me through how you designed an API at TCS.

Answer: JAX-RS is the Java EE standard for building RESTful web services using annotations like `@GET`, `@POST`, `@Path`, `@Produces`, and `@Consumes`. In Spring Boot this is implemented via Spring MVC's `@RestController`. At TCS I designed an API to fetch complete historical policy details. The design involved: defining the resource path as `/api/policies/{policyId}/history`, accepting query parameters for date ranges, validating inputs at the controller layer, calling a service layer that executed JDBC queries, and returning a structured JSON response. I paid attention to HTTP status codes — 200 for success, 404 when a policy wasn't found, 400 for invalid parameters. I also ensured the API was stateless so multiple instances could serve requests without session affinity, which matters for scalability in a BaNCS-like multi-server deployment.

■ *Interviewer tip: They may ask for a live design — practice drawing this on a whiteboard.*

Q10. How do you ensure thread safety in a Java EE application under high concurrency?

Answer: Thread safety in Java EE starts with understanding that Servlets and Spring controllers are singletons by default — so any instance variable shared across requests is a thread-safety risk. My approach: keep controller and service beans stateless, pass state through method parameters. For shared mutable state, use synchronized blocks, `ReentrantLock`, or `java.util.concurrent` data structures like `ConcurrentHashMap`. In connection pooling (HikariCP in Spring), connections are managed per-thread, which handles DB concurrency safely. In my Kafka-based Lovable Clone, I ensured consumer groups were designed so each partition is consumed by one thread, avoiding concurrent access to the same message partition.

Q11. Explain the Java EE security model — how did you handle authentication/authorization in BaNCS or Health Transformer?

Answer: Java EE security covers authentication (who are you) and authorization (what can you do) via JAAS, Servlet security constraints, EJB method-level security (`@RolesAllowed`), and HTTPS for transport security. In BaNCS and Health Transformer, authentication was handled at the platform/infrastructure level — users authenticated via the BaNCS login mechanism which passed identity context to backend services. Within the application, role-based access was enforced at the API layer — Spring Security with role checks ensured that, for example, only claims processors could access claims APIs. Channel ID-specific configurations in Health Transformer also acted as a form of authorization — ensuring that operations from a specific channel (like direct vs broker) followed the correct business rules and data visibility constraints.

Q12. What is the difference between `@Stateless`, `@Stateful`, and `@Singleton` EJBs? When would you choose each?

Answer: `@Stateless` EJBs hold no client state — each method call is independent. They're pooled by the container for performance and ideal for most business logic like policy lookups or payment calculations. `@Stateful` EJBs maintain conversational state per client session — useful for multi-step wizards like a complex policy enrollment flow where partial state must persist across calls. `@Singleton` EJBs are instantiated once per application and are suitable for shared resources like a configuration cache or a connection to an external system. In an insurance context I would use `@Stateless` for claims processing services, `@Stateful` for a multi-step underwriting questionnaire, and `@Singleton` for caching product configuration data that is loaded once at startup.

3 · Insurance Platform Knowledge (BaNCS / Majesco / DxC)

Q13. Compare TCS BaNCS architecture with what you know of Majesco or DxC.

Answer: TCS BaNCS is a monolithic-to-SOA platform built on Java/Oracle, strong in BFSI and health insurance, with deep configurability for policy, claims, and billing. It uses a layered Java EE architecture with stored procedures for data-intensive operations. Majesco (now Majesco Cloud) is more cloud-native, offering microservices-oriented architecture with APIs-first design, better suited for greenfield SaaS deployments. DxC (formerly CSC) platforms like Vantage or Duck Creek tend to be .NET-based with strong P&C insurance focus and agent portal integration. The key difference is modernization posture: BaNCS suits large established carriers with existing Oracle investments; Majesco suits carriers wanting cloud-native transformation. My BaNCS experience gives me insight into both the business domain and the architectural patterns common across all these platforms — policy lifecycle, product configurability, and claims workflow.

■ *Interviewer tip: Research Majesco briefly before the interview — knowing 2-3 key differentiators will impress.*

Q14. How is a policy lifecycle typically modeled in BaNCS? Walk through states from issuance to claims settlement.

Answer: A policy lifecycle in BaNCS moves through distinct states: Quote → Application → Underwriting Review → Policy Issued → Active → Endorsement (if changes needed) → Renewal → Lapse/Cancellation → Claims Intimation → Claims Processing → Claims Settlement → Closed. Each state transition triggers workflow tasks and data operations. In Health Transformer, I worked on the Policy module which managed transitions from issuance through active maintenance. The Workflow module orchestrated task assignments at each state change — for example, routing a claim to the appropriate adjuster based on claim type and amount. BaNCS uses configurable product setups so the lifecycle can vary by product line. Channel ID configurations in Health Transformer controlled which channels could initiate which lifecycle transitions.

Q15. In Health Transformer, how were Claims and Policy modules integrated? What were the key data flows?

Answer: Claims and Policy modules in Health Transformer shared a common data foundation — the policy record. When a claim was intimated, the Claims module fetched the active policy details to verify coverage, check benefit limits, and determine applicable deductibles. This cross-module data access was done via service calls (Java/Spring) rather than direct DB joins to maintain module separation. The Workflow module sat above both — it tracked the claim's processing state and assigned tasks to users. Key data flows: member enrolls → Policy module creates master policy record → Claim raised against policy → Claims module reads policy coverage → validates against business rules → routes to Workflow for approval → settlement updates claims and billing records. I contributed to maintaining these flows, fixing defects where data mismatches caused incorrect benefit calculations.

Q16. What challenges did you face with the EDI module defects, and how did you resolve them architecturally?

Answer: EDI (Electronic Data Interchange) in Health Transformer handled inbound and outbound transactions with external parties like TPAs, hospitals, and regulatory bodies. The defects I resolved were primarily around parsing failures — incoming EDI messages with unexpected field formats or segment variations were causing processing errors. My resolution approach involved: first understanding the EDI

transaction set (like 834 for enrollment or 837 for claims), then tracing the parsing logic in Java to identify where the assumption about format was breaking, then adding defensive parsing with proper error handling and logging. Architecturally, I recommended adding a validation layer before the main processing pipeline so malformed EDI messages are caught early, logged with context, and routed to an error queue rather than failing silently mid-process.

Q17. How does BaNCS handle multi-channel distribution? Did you work on Channel ID-specific requirements?

Answer: BaNCS supports multi-channel distribution through configurable Channel IDs — each channel (direct, broker, agent, bank assurance) gets a unique identifier that drives business rule variations. Different channels may have different product offerings, commission structures, data capture requirements, or workflow routing. Yes — in Health Transformer I worked directly on Channel ID-specific requirements. I collaborated with Business Analysts to understand what each channel required, then implemented conditional logic in the backend where Channel ID determined which business rules to apply. For example, a broker channel might require additional disclosure fields on the policy that a direct channel does not. This was implemented using a strategy-like pattern where channel-specific handlers were invoked based on the Channel ID.

Q18. Explain how workflow engines are used in core insurance platforms.

Answer: Workflow engines in insurance platforms manage the sequencing, routing, and tracking of tasks across the policy and claims lifecycle. They define: who does what, in what order, under what conditions. In BaNCS/Health Transformer, the Workflow module handled task creation and assignment — for example, after a claim is submitted, a workflow task is created for a claims examiner; once approved, another task goes to the finance team for payment. Workflow engines typically support parallel paths (e.g., medical review and legal review happening simultaneously), escalation rules (task not completed in X hours escalates to supervisor), and SLA tracking. I supported the Workflow module by fixing defects where task routing was incorrect — tasks going to wrong queues or not triggering downstream steps after completion.

Q19. What are common integration patterns when connecting a core insurance platform to external systems?

Answer: Common patterns include: API-based integration (REST/SOAP) for real-time queries like hospital network lookups or payment gateway calls; EDI for bulk batch transactions with TPAs and regulators; event-driven messaging (JMS/Kafka) for asynchronous operations like claim status notifications; file-based integration (SFTP) for legacy system compatibility. In BaNCS, external system calls were often wrapped in adapter layers to insulate core business logic from partner-specific protocols. I applied this in Health Transformer where the EDI module acted as an adapter translating between the platform's internal model and the EDI transaction sets. For new integrations I would recommend an API gateway pattern with retry logic, circuit breakers, and dead-letter queues for resilience.

4 · Architecture Design & Scalability

Q20. How would you design a scalable policy administration system for 1 million policies with peak renewal season traffic?

Answer: I would design this as a horizontally scalable microservices system. Core services: Policy Service, Renewal Engine, Notification Service, and Billing Service. For the renewal peak: separate the renewal calculation (CPU-intensive) from notification dispatch (I/O-intensive) using an event-driven approach with Kafka — the Renewal Engine publishes renewal events, Notification Service consumes asynchronously. Use Redis for caching policy summary data to avoid repeated DB reads for the same policies during peak. Database: partition the policy table by renewal month so peak queries hit a subset of data. Use connection pooling (HikariCP) and read replicas to distribute DB load. Deploy behind a load balancer with auto-scaling groups. For the 1 million policy scale, the Renewal Engine would batch-process policies in chunks using parallel streams or worker threads, publishing Kafka events at controlled throughput.

Q21. Walk me through the architecture of your Lovable Clone project.

Answer: Lovable Clone is a platform that lets users generate web applications through a conversational interface — similar to Lovable.dev. The backend is built with Java Spring Boot using a Microservices architecture. Services include: a Gateway Service for routing, a Generation Service for handling AI prompt processing, a Project Service for managing user projects, and a Storage Service for persisting generated code. Apache Kafka enables asynchronous communication — when a user submits a generation request, it's published to a Kafka topic; the Generation Service consumes it, processes the request, and publishes the result to a response topic. Redis caches recently generated project data and user session state to reduce DB hits. PostgreSQL stores persistent project and user data with proper schema design and indexing. Docker containers ensure consistent deployment across environments. The architecture prioritises decoupling and scalability — each service can be scaled independently based on load.

Q22. When would you choose microservices over a monolithic architecture for an insurance core platform?

Answer: Microservices make sense when: the system needs to scale different components independently (claims processing spikes during catastrophe events while policy admin remains steady), teams are large enough to own separate services without coordination overhead, and deployment agility is critical. Monoliths make sense for smaller teams, tight budgets, or when the domain isn't well-understood yet — BaNCS is essentially a modular monolith which works well for carriers who want a single vendor, integrated platform. For insurance platform modernisation I'd recommend a strangler fig approach: keep the core monolith running, extract high-value services (like real-time quoting or claims intake) as microservices first, validate the pattern, then progressively decompose. This reduces risk compared to a big-bang rewrite.

Q23. Explain the CQRS pattern. Where would you apply it in a claims processing system?

Answer: CQRS (Command Query Responsibility Segregation) separates write operations (commands) from read operations (queries) using different models. The write model handles business logic and enforces invariants; the read model is optimised for query performance, often denormalised. In claims processing: the write side handles ClaimSubmit, ClaimApprove, ClaimReject commands — each updating the claims aggregate with full business rule enforcement. The read side maintains a ClaimsSummary

projection — a denormalised view joining claim, policy, and member data — optimised for adjuster dashboards and reporting queries. When a claim state changes, an event updates the read model asynchronously. This means the dashboard never hits the normalised write database, improving query performance significantly at scale.

Q24. How does the Saga pattern help manage distributed transactions? Give an insurance example.

Answer: The Saga pattern manages long-running distributed transactions through a sequence of local transactions, each publishing events or messages to trigger the next step. If a step fails, compensating transactions undo previous steps. Insurance example — Policy Issuance Saga: (1) Premium Calculation Service calculates and reserves premium, (2) Payment Service processes first payment, (3) Policy Service activates the policy, (4) Notification Service sends welcome documents. If Payment fails at step 2, a compensating transaction releases the premium reservation at step 1. This avoids a distributed lock across all services. I would implement this as a choreography-based saga using Kafka — each service listens for its trigger event and publishes success/failure events — which aligns with the Lovable Clone's Kafka-based event flow I've already built.

Q25. How would you design for idempotency in a payment or claims API?

Answer: Idempotency ensures that retrying a request produces the same result as the first call. For a claims payment API: require clients to send an idempotency key (UUID) in the request header. On the server, before processing, check a Redis cache (or idempotency table in DB) for this key. If found, return the cached response immediately without reprocessing. If not found, process the request, store the result against the key with a TTL (e.g., 24 hours), and return the response. This protects against double payments when a network timeout causes a client to retry. In BaNCS-style systems, this is critical because payment gateway integrations are inherently unreliable — retries are common and double-processing a claim payment has serious financial consequences.

Q26. What caching strategies did you use with Redis in your personal project? How would you apply them in enterprise insurance?

Answer: In Lovable Clone I used Redis for: cache-aside pattern for project data (application checks cache first, falls back to DB, populates cache on miss), and session caching for user context. TTL-based expiry ensured stale data didn't persist. In an enterprise insurance context I'd apply: policy summary caching (policy header data read thousands of times daily but changes infrequently), product configuration caching (product rules loaded at startup, refreshed on change events), and rate limiting (Redis-based counters to throttle API calls per channel). For claims, I'd be more conservative — claims data changes frequently so caching needs careful invalidation strategies, ideally event-driven invalidation when a claim state changes.

5 · Integration Patterns & Middleware

Q28. What integration patterns are most relevant to insurance platforms?

Answer: Key patterns: (1) Event-driven/Publish-Subscribe — claims events broadcast to notification, billing, and reporting services simultaneously. (2) Request-Reply — synchronous calls for real-time premium calculation or eligibility checks. (3) Dead Letter Queue — failed EDI messages or payment notifications parked for manual review rather than lost. (4) Idempotent Consumer — ensuring the same EDI transaction isn't processed twice. (5) Aggregator — combining data from policy, member, and claims services into a single claims dashboard response. (6) Saga/Process Manager — orchestrating multi-step policy issuance. In my TCS work the EDI module used a pipeline pattern — inbound EDI flows through a validation stage, transformation stage, and processing stage sequentially.

Q29. How did you use Apache Kafka in your Lovable Clone? How would it benefit a real-time claims notification system?

Answer: In Lovable Clone, Kafka decouples the generation request intake from the actual AI processing. A user's request is published to a 'generation-requests' topic immediately, giving instant acknowledgement. The Generation Service consumes at its own pace, handles backpressure naturally, and publishes results to a 'generation-results' topic. This prevents the API from blocking on long-running AI calls. For real-time claims notifications: when a claim status changes (submitted, approved, paid), a `ClaimStatusChanged` event is published to Kafka. Multiple consumers react: Notification Service sends SMS/email, Analytics Service updates dashboards, TPA Integration Service notifies the third-party administrator. Each consumer processes independently and at its own pace — adding a new consumer (e.g., regulator reporting) requires no changes to the claims processing service itself.

Q30. Explain the difference between synchronous REST and asynchronous messaging. When do you use each?

Answer: Synchronous REST: caller waits for a response. Use when: you need an immediate answer (premium quote, eligibility check, user-facing API calls). Benefits: simple, request-response contract, easy error propagation. Risks: caller is blocked, downstream slowness cascades. Asynchronous messaging: caller publishes a message and continues. Use when: downstream processing takes time (claims adjudication, document generation), you need decoupling, or multiple consumers need the same event. Benefits: resilience, scalability, natural load levelling. In insurance: policy query APIs are synchronous; claims processing pipeline, renewal batch, and notification dispatch are asynchronous. My Lovable Clone mixed both — the intake API is synchronous REST, the generation pipeline is async Kafka.

Q32. How do you handle API versioning and backward compatibility in a long-lived insurance platform?

Answer: Three main strategies: (1) URI versioning — `/api/v1/policies` vs `/api/v2/policies`. Simple, explicit, easy to route. (2) Header versioning — version in Accept header. Cleaner URLs but harder to test in browsers. (3) Parameter versioning — least preferred. For backward compatibility: add fields to responses rather than remove them, never change field types, deprecate old versions with a sunset header giving consumers migration time. In BaNCS-style long-lived platforms this is critical — downstream systems (TPAs, broker portals, internal tools) may be on old versions for years. I would maintain at least N-1 versions in production and document breaking changes thoroughly. For internal microservices I prefer event schema versioning with Avro/protobuf to enforce compatibility at the schema registry level.

6 · Performance & Troubleshooting

Q34. Walk me through a production issue you resolved post-conversion at TCS.

Answer: After a server migration, policy history queries started returning partial data for certain policy types. I began root cause analysis by comparing the query execution plans on old and new environments using Oracle's EXPLAIN PLAN. I found that a legacy PL/SQL procedure was relying on an index that existed on the old database but hadn't been recreated on the new one — causing a full table scan and incomplete result sets due to a timeout. The immediate fix was creating the missing index. The longer-term fix was rewriting the procedure in Java using explicit JDBC PreparedStatements with parameterised queries that didn't depend on implicit index hints. I also added monitoring to alert on query execution times exceeding thresholds. This experience shaped how I approach migration architecture — always validate database objects, indexes, and environment-specific configurations explicitly rather than assuming they transfer.

Q35. How do you diagnose a memory leak in a Java EE application in production?

Answer: Memory leaks in Java typically manifest as increasing heap usage over time until OutOfMemoryError. Diagnosis steps: (1) Monitor heap with JConsole or Prometheus/Grafana — confirm heap grows without release after GC. (2) Capture heap dump using jmap or JVM flags (-XX:+HeapDumpOnOutOfMemoryError). (3) Analyse with Eclipse MAT or VisualVM — look for objects with unexpectedly high retained heap. Common causes in Java EE: static collections accumulating objects, unclosed JDBC connections or ResultSets (critical in JDBC-heavy systems like mine), ThreadLocal variables not cleaned up, or cache growing without eviction. In my JDBC work I always used try-with-resources to ensure Connection, PreparedStatement, and ResultSet were closed, which is the most common leak point in JDBC-heavy applications.

Q37. How would you optimize a slow database query in a high-volume claims processing system?

Answer: Step 1: Get the execution plan (EXPLAIN PLAN in Oracle) — identify full table scans, nested loops on large tables, or missing index usage. Step 2: Add appropriate indexes — composite indexes for multi-column WHERE clauses, covering indexes for queries that select only indexed columns. Step 3: Rewrite the query — avoid SELECT *, use EXISTS instead of IN for correlated subqueries, avoid functions on indexed columns in WHERE clauses. Step 4: Consider partitioning for very large tables — claims table partitioned by claim date means queries for recent claims hit only recent partitions. Step 5: Cache frequently-read reference data (procedure codes, ICD codes) in Redis. In BaNCS I applied query optimisation regularly — Oracle's PL/SQL profiler helped identify the exact lines consuming most execution time.

Q38. How does connection pooling work? What parameters do you tune under heavy load?

Answer: Connection pooling maintains a pool of pre-established database connections that are reused across requests, avoiding the overhead of creating a new connection per request. HikariCP (default in Spring Boot) manages this pool. Key parameters under heavy load: maximumPoolSize — max concurrent connections (set based on DB server capacity, typically cores × 2 + effective_spindle_count); minimumIdle — connections kept warm; connectionTimeout — how long to wait for a connection before throwing an error; idleTimeout — when to retire idle connections; maxLifetime — recycle connections to avoid stale connections or DB-side limits. Under sustained high load I'd also monitor pool wait time metrics — if

threads are frequently waiting for connections, either increase pool size or optimise queries to release connections faster.

7 · SDLC, Architectural Frameworks & Design

Q40. Which architectural frameworks or reference models have you used or studied?

Answer: I'm familiar with the 4+1 View Model — using logical, process, physical, development, and scenario views to describe architecture from different stakeholder perspectives. I've studied TOGAF at a conceptual level — understanding the ADM (Architecture Development Method) cycle from Architecture Vision through Migration Planning. In practice at TCS, we used a pragmatic approach: documenting architecture via component diagrams, sequence diagrams, and deployment diagrams using UML rather than formal TOGAF artifacts. For Lovable Clone I created explicit architecture decision records (ADRs) — documenting why I chose Kafka over RabbitMQ, PostgreSQL over MySQL — which is a practice I'd bring to a formal architect role.

Q43. How do you approach non-functional requirements (NFRs) like availability, reliability, and security during design?

Answer: NFRs must be defined as measurable targets, not vague aspirations. For an insurance platform: Availability — 99.9% uptime (less than 9 hours downtime/year), designed with active-active deployment and DB failover. Performance — policy query API responds in under 500ms at the 95th percentile under 200 concurrent users. Security — all PII encrypted at rest (AES-256) and in transit (TLS 1.2+), OWASP Top 10 mitigations applied, API authentication via OAuth2/JWT. Reliability — circuit breakers on external integrations, dead-letter queues for failed async messages. I address NFRs during the design phase, not as an afterthought — each architectural decision is evaluated against how it helps or hinders the NFR targets. In Health Transformer, performance requirements drove the decision to use JDBC over ORM for bulk data operations.

Q44. Walk me through how you'd conduct a technical design review for a new feature on an insurance platform.

Answer: A design review has phases: (1) Requirements alignment — confirm the feature's functional and non-functional requirements are clearly stated and agreed. (2) Design walkthrough — the author presents the proposed solution covering data model changes, API contracts, integration points, and sequence flows. (3) NFR assessment — can the design meet performance, security, and scalability targets? (4) Risk identification — what could go wrong? Are there migration risks, backward compatibility issues, or external dependencies? (5) Alternative evaluation — are there simpler approaches? (6) Decision and documentation — capture decisions and rationale in an ADR. At TCS I participated in design discussions for Channel ID implementations and claims defect resolution, contributing both technical analysis and implementation suggestions.

8 · Cloud, Docker & DevOps

Q45. How did you use Docker in your personal project? How does containerisation change deployment architecture?

Answer: In Lovable Clone, Docker containerised each microservice — the Gateway, Generation Service, Project Service, and supporting infrastructure (Kafka, Redis, PostgreSQL) — into independent containers defined in a `docker-compose.yml`. This guaranteed that the application ran identically on my development machine, a test environment, and production. Architecturally, containerisation enables: infrastructure-as-code (the compose file IS the deployment spec), horizontal scaling (run 3 instances of the Generation Service behind a load balancer), environment parity (no more 'works on my machine'), and fast rollback (previous container image is always available). For an enterprise insurance platform migration, containerising services with Docker and orchestrating with Kubernetes enables gradual modernisation — run legacy BaNCS components alongside new microservices in the same cluster.

Q47. How would you migrate a BaNCS-based on-premise system to a cloud-native architecture?

Answer: I'd use the strangler fig pattern over a big-bang rewrite. Phase 1: Lift and shift — move BaNCS to cloud VMs first, get stable, establish cloud networking and security baseline. Phase 2: Extract high-value services — identify modules like real-time quoting or claims intake that benefit most from cloud elasticity. Build these as cloud-native microservices with REST APIs, deploy on containers (EKS/AKS), and route traffic from the monolith via API gateway. Phase 3: Data modernisation — migrate Oracle schemas to Aurora PostgreSQL or managed Oracle, implement CDC (Change Data Capture) to sync data during transition. Phase 4: Progressive decomposition — continue extracting services. Key risks to manage: data consistency during dual-write periods, Oracle-specific SQL that doesn't migrate cleanly, and integration contracts with external TPAs that expect specific message formats.

9 · Leadership & Cross-Functional Collaboration

Q49. Describe a time you had to align technical architecture with a seemingly contradictory business requirement.

Answer: In Health Transformer, the business required Channel ID-specific policy variations — different fields, rules, and workflows per channel — but also wanted a single unified policy processing codebase for maintainability. These goals initially felt contradictory. I proposed a strategy pattern-based solution: a common policy processing pipeline with channel-specific handlers injected at defined extension points. The business got their channel variations; the team got a single codebase with predictable extension points rather than scattered if/else logic. I presented this to the BA team with sequence diagrams showing how each channel's flow would work, which helped them validate that business rules were correctly captured. The alignment came from translating business concepts (channel behaviour) into architectural patterns (strategy/plugin) that both sides could reason about.

Q50. How have you mentored or guided junior developers? Give a specific example.

Answer: At TCS I guided junior team members on JDBC best practices — specifically around resource management. I noticed a pattern where junior developers were not closing JDBC resources in finally blocks (pre-Java 7 style), risking connection leaks. I organised a brief knowledge-sharing session demonstrating try-with-resources and why it matters — showing with a simple benchmark how connection pool exhaustion manifests. I also reviewed pull requests and left detailed comments explaining not just what to fix but why — connecting the code change to the architectural principle. The goal was to build architectural awareness, not just fix individual bugs. I believe knowledge transfer is a force multiplier and I plan to continue this at Accenture.

Q52. Tell me about a time you recommended adoption of a new technology. How did you build the case?

Answer: For Lovable Clone I recommended Apache Kafka over a simpler queue (like RabbitMQ) for inter-service messaging. The case: (1) Message replay — Kafka retains messages for configurable periods, allowing a new consumer service to process historical events without re-triggering them. This matters in insurance for audit trails. (2) Consumer group scaling — multiple consumer instances in a group naturally parallelise processing across partitions. (3) Event sourcing readiness — Kafka's log structure enables event sourcing patterns. I built a simple comparison table showing Kafka vs RabbitMQ on: throughput, replay capability, consumer flexibility, and operational complexity. The tradeoff was higher operational complexity, which I offset by using a managed Kafka (Confluent) for production. This structured approach — articulate the need, compare options, acknowledge tradeoffs — is how I'd build the case in a client engagement.

10 · Scenario & Case Study Questions

Q53. A client wants to migrate their legacy mainframe-based policy system to Java EE microservices. How would you approach the architecture?

Answer: Step 1: Discovery — understand the mainframe's data model, business rules (often encoded in COBOL), and integration landscape (batch feeds, terminal sessions, external interfaces). Step 2: Domain decomposition — map the mainframe modules to bounded contexts: Policy, Billing, Claims, Product Catalogue. Each becomes a candidate microservice. Step 3: Strangler fig — build a Java EE API gateway in front of the mainframe. New business features go into Java microservices; mainframe serves existing ones. Use event sourcing or CDC to sync data between new services and mainframe during transition. Step 4: Data migration — migrate the Oracle/DB2 schema to PostgreSQL or managed Oracle, validate with dual-write and reconciliation. Step 5: Decommission — once all traffic routes through Java services and data is validated, retire the mainframe module. Key risks: COBOL business logic that isn't documented (must be reverse-engineered from test cases), batch job dependencies, and performance expectations set by mainframe throughput.

Q54. Claims processing is experiencing intermittent failures during peak hours. Walk me through your investigation.

Answer: Step 1: Reproduce and scope — when does it fail? What percentage of claims? Which claim types? Correlate with time-of-day metrics to confirm peak correlation. Step 2: Check infrastructure metrics — CPU, memory, DB connection pool utilisation, network latency during peak. HikariCP pool exhaustion (threads waiting for connections) is a common culprit. Step 3: Analyse application logs — look for timeout exceptions, connection refused errors, or slow query warnings. Step 4: Profile DB — slow query log during peak. Are there lock contentions? Long-running transactions blocking others? Step 5: Check external dependencies — is a TPA or payment gateway response time degrading under load, causing upstream timeouts? Step 6: Short-term fix based on findings (increase pool size, add index, implement circuit breaker). Long-term: consider making claims intake asynchronous with Kafka — accept claims to a queue immediately, process asynchronously, eliminating synchronous timeout risk.

Q55. Design a real-time premium calculation engine handling 500 concurrent quote requests with sub-second response times.

Answer: Architecture: stateless Spring Boot microservice deployed in 3+ instances behind a load balancer. No DB calls on the hot path — product rules and rate tables loaded into Redis at startup, refreshed via a configuration change event. Calculation logic: pure Java with no I/O — parameterised input goes through actuarial formula engine, result returned in memory. For complex products with many calculation steps, use parallel streams for independent sub-calculations. Cache quote results with a short TTL (5 minutes) using the input parameters hash as cache key — repeat quote requests from the same user hit Redis, not the calculation engine. Monitor: P95 and P99 latency with Micrometer/Prometheus. Target: P99 under 200ms for standard products, 500ms for complex (multi-rider health plans). Scale: each stateless instance can handle ~150-200 concurrent requests, so 3 instances comfortably cover 500 concurrent.

11 · HR & Fitment Questions

Q57. Why Accenture over other companies?

Answer: Accenture's scale of insurance and BFSI engagements is unmatched — the breadth of client exposure across carriers, reinsurers, and InsurTech companies is something I can't replicate elsewhere at this stage. Specifically, this role's focus on Java EE and insurance core platforms (Majesco, DxC, BaNCS) is a direct match for my TCS background — I won't spend the first year learning the domain, I'll be contributing from day one. Beyond the technical fit, Accenture's investment in cloud and AI-driven insurance transformation — areas I'm building skills in through projects like Lovable Clone — aligns with where I want to grow as an architect over the next few years. Mumbai specifically is where I want to build my career.

Q58. Why are you looking to move from TCS after 3.3 years?

Answer: TCS gave me an exceptional foundation — real enterprise platform experience, production responsibility, and cross-domain exposure in BFSI and health insurance. What I'm seeking now is the opportunity to move into a formal architecture role where I'm designing systems end-to-end rather than contributing to a larger team's execution. The Technology Architect role at Accenture is that step. I also want exposure to multiple clients and platforms — having worked deeply on BaNCS, I want to understand how Majesco and DxC solve the same problems differently, which a consulting environment enables. It's not about leaving TCS — it's about the right next step in an architect trajectory.

■ *Interviewer tip: Never speak negatively about TCS — frame it as progression, not escape.*

Q59. Where do you see yourself in 3 years as a Technology Architect?

Answer: In 3 years I see myself leading architecture for a large-scale insurance platform engagement — either a core system modernisation or a cloud migration for a major carrier. I want to have hands-on experience across at least two insurance platforms (adding Majesco or DxC to my BaNCS background), and to have led a team of developers technically — establishing coding standards, architecture review processes, and knowledge transfer practices. I'm also investing in cloud architecture (AWS/Azure) and event-driven systems, so I'd like to have delivered a cloud-native insurance component in production. The goal is to be the person clients and Accenture leadership bring in when a complex insurance transformation needs credible technical direction.

Q60. Are you comfortable with the Mumbai location and client-facing responsibilities?

Answer: Yes — Mumbai is exactly where I want to be. The concentration of financial services and insurance companies makes it the right city for a career in this domain. On client-facing responsibilities: I've collaborated with Business Analysts and stakeholders at TCS to translate requirements into technical solutions, which is foundational client-facing work. I'm comfortable presenting technical designs, asking clarifying questions, and managing expectations around delivery. Moving into a more formal client-facing architect role is actually a goal — the ability to bridge business and technology is what distinguishes an architect from a senior developer, and I'm actively developing that skill.

Priority Cheat Sheet — Last Minute Review

■ Critical	Java EE internals (transactions, JDBC, thread safety), BaNCS policy lifecycle, REST API design, CQRS, Saga, idempotency
■ Important	Kafka/Redis from Lovable Clone, EDI defects story, post-conversion RCA story, scalability design answers
■ Good to have	TOGAF awareness, Majesco vs DxC comparison, cloud migration strategy, Docker/K8s concepts

Best of luck, Amit! You have the right experience — tell your story with confidence. ■